

Advanced Programming Techniques

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Prof. Dr. Michael Lipp

Character Encoding

Characters in memory

- **Computer memory...**
 - ... stores groups of bits
 - ... cannot store “characters”
- **Workaround**
 - Map particular groups of bits (interpreted as unsigned values) to characters
- **Historically, there was a big mess...**

ASCII

- **Standardized and popular: ASCII (American standard code for information interchange)**
- **Built-in in C++**
 - Writing 'a' actually causes the compiler to lookup the value associated with character a in the ASCII table and work with this value.

ASCII Table

Oct	Dec	Hex	Char
000	0	00	NUL '\0' (null character)
001	1	01	SOH (start of heading)
002	2	02	STX (start of text)
003	3	03	ETX (end of text)
004	4	04	EOT (end of transmission)
005	5	05	ENQ (enquiry)
006	6	06	ACK (acknowledge)
007	7	07	BEL '\a' (bell)
010	8	08	BS '\b' (backspace)
011	9	09	HT '\t' (horizontal tab)
012	10	0A	LF '\n' (new line)
013	11	0B	VT '\v' (vertical tab)
014	12	0C	FF '\f' (form feed)
015	13	0D	CR '\r' (carriage ret)
016	14	0E	SO (shift out)
017	15	0F	SI (shift in)
020	16	10	DLE (data link escape)
021	17	11	DC1 (device control 1)
022	18	12	DC2 (device control 2)
023	19	13	DC3 (device control 3)
024	20	14	DC4 (device control 4)
025	21	15	NAK (negative ack.)
026	22	16	SYN (synchronous idle)
027	23	17	ETB (end of trans. blk)
030	24	18	CAN (cancel)
031	25	19	EM (end of medium)
032	26	1A	SUB (substitute)
033	27	1B	ESC (escape)
034	28	1C	FS (file separator)
035	29	1D	GS (group separator)
036	30	1E	RS (record separator)
037	31	1F	US (unit separator)

Oct	Dec	Hex	Char
040	32	20	SPACE
041	33	21	!
042	34	22	"
043	35	23	#
044	36	24	\$
045	37	25	%
046	38	26	&
047	39	27	'
050	40	28	(
051	41	29	)
052	42	2A	*
053	43	2B	+
054	44	2C	,
055	45	2D	-
056	46	2E	.
057	47	2F	/
060	48	30	0
061	49	31	1
062	50	32	2
063	51	33	3
064	52	34	4
065	53	35	5
066	54	36	6
067	55	37	7
070	56	38	8
071	57	39	9
072	58	3A	:
073	59	3B	;
074	60	3C	<
075	61	3D	=
076	62	3E	>
077	63	3F	?

Oct	Dec	Hex	Char
100	64	40	@
101	65	41	A
102	66	42	B
103	67	43	C
104	68	44	D
105	69	45	E
106	70	46	F
107	71	47	G
110	72	48	H
111	73	49	I
112	74	4A	J
113	75	4B	K
114	76	4C	L
115	77	4D	M
116	78	4E	N
117	79	4F	O
120	80	50	P
121	81	51	Q
122	82	52	R
123	83	53	S
124	84	54	T
125	85	55	U
126	86	56	V
127	87	57	W
130	88	58	X
131	89	59	Y
132	90	5A	Z
133	91	5B	[
134	92	5C	\ '\\
135	93	5D	]
136	94	5E	^
137	95	5F	~
140	96	60	`

Oct	Dec	Hex	Char
140	96	60	`
141	97	61	a
142	98	62	b
143	99	63	c
144	100	64	d
145	101	65	e
146	102	66	f
147	103	67	g
150	104	68	h
151	105	69	i
152	106	6A	j
153	107	6B	k
154	108	6C	l
155	109	6D	m
156	110	6E	n
157	111	6F	o
160	112	70	p
161	113	71	q
162	114	72	r
163	115	73	s
164	116	74	t
165	117	75	u
166	118	76	v
167	119	77	w
170	120	78	x
171	121	79	y
172	122	7A	z
173	123	7B	{
174	124	7C	
175	125	7D	}
176	126	7E	~
177	127	7F	DEL

Extended character sets

- Problem with ASCII: no “national characters” such as “äöüß”
- Obvious (but insufficient) solution: use unused 128 numbers
 - Internationally, there are more than 128 “national characters”
 - Workaround: code pages
 - ISO:
 - ISO-8859-1: Latin-1 Supplement
 - ISO-8859-15: Latin alphabet No. 9 (Euro “€”!)
 - ...
 - Windows:
 - 1250: Latin 2 / Central European
 - 1252: Latin 1 / Western European
 - 1253: Greek
 - ...

Final solution: Unicode

- **Defines**
 - 137,766 graphic characters
 - 163 format characters
 - 65 control characters
- **Organized in Planes (16-bit each)**
 - Most important: Basic Multilingual Plane (BMP)
 - Characters for almost all modern languages and a large number of symbols
- **Encodings**
 - UTF-32 (selects any character)
 - UTF-16 (selects character on BMP, “Escape” to select any character)
 - UTF-8 (selects any character)



Source: <https://shop.designinmainz.de/produkt/decodeunicode-basic-multilingual-plane-bmp-map/>

UTF-8

- Variable width encoding

Number of bytes	Bits for code point	First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
1	7	U+0000	U+007F	0xxxxxxx			
2	11	U+0080	U+07FF	110xxxxx	10xxxxxx		
3	16	U+0800	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx	
4	21	U+10000	U+10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

Source: Wikipedia

- Same as ASCII for the first 128 values
- Default on most Linux systems
- Supported (but not default) in Windows 10

Converters

- **Standard Library**

- `codecvt` (see <http://www.cplusplus.com>)
 - Hard to use
 - “The entire header `<codecvt>` (which does not contain the class `codecvt`!) is deprecated, as are the utilities `wstring_convert` and `wbuffer_convert`. These features are hard to use correctly, and there are doubts whether they are even specified correctly. Users should use dedicated text-processing libraries instead.” (<https://isocpp.org/files/papers/p0636r0.html>)
- Streams can be imbued with a `codecvt`, which makes them convert automatically

Poco

- **Poco TextConverter**

- Works together with TextEncoding

- Basic usage:

```
string text = "\u00e4\u00f6\u00fc";  
cout << text.size() << " " << text << endl;  
Poco::UTF8Encoding utf8enc;  
Poco::Windows1252Encoding winEnc;  
Poco::TextConverter conv(utf8enc, winEnc);  
std::string converted;  
conv.convert(text, converted);  
cout << converted.size() << " " << converted << endl;
```